

LECTURE 16

Classical Synchronization Problems II

CSC323 – Operating Systems

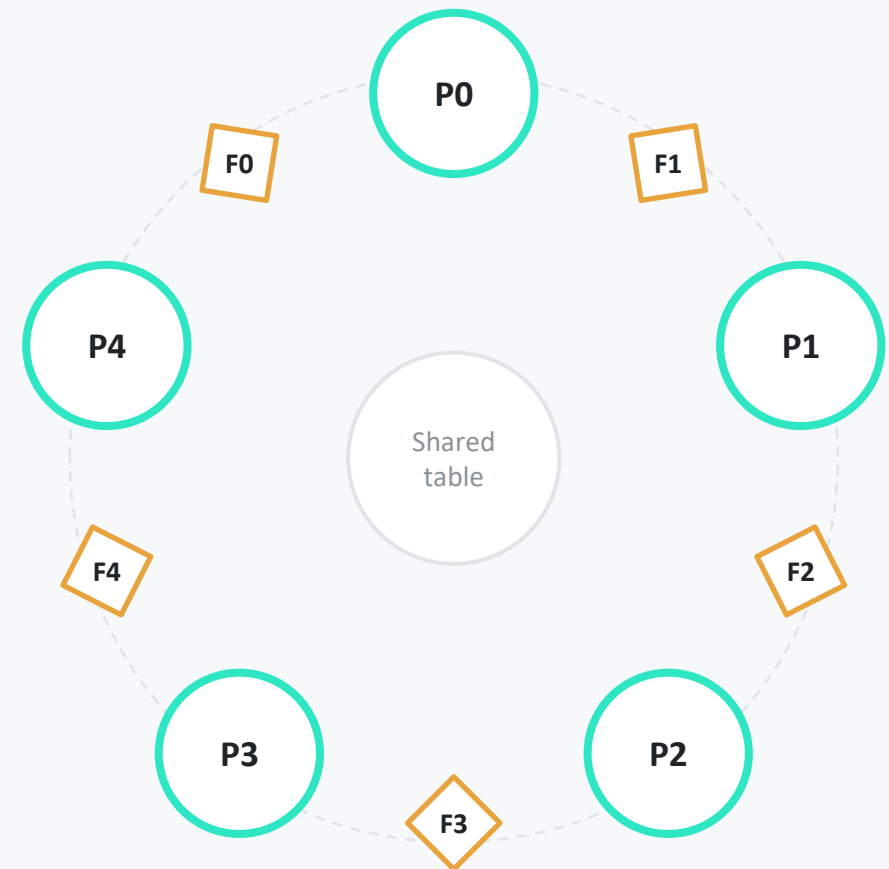
COMSATS University Islamabad, Lahore Campus

In This Lecture

- The Dining Philosophers Problem's setup, and what it's really a model of
- The naive solution, and a precise trace of exactly how it can deadlock
- An asymmetric fix that breaks the circular-wait pattern without any extra bookkeeping
- The monitor-based solution — `state[]` and `test()` — and why it expresses the problem's real constraint more cleanly than raw semaphores can
- A short case study: Windows dispatcher objects and `WaitForSingleObject()`, and Linux `futexes` and `pthread`s

The Dining Philosophers Problem

- Five philosophers sit at a circular table. Between each adjacent pair sits exactly one fork, for five forks total.
- Each philosopher alternates between thinking (needs nothing) and eating (needs both adjacent forks, picked up one at a time).



A Model of Fixed Resources, Multiple Needed at Once

- The setup is a clean model of a very general OS problem: several processes contending for a fixed pool of shared resources, where each process needs more than one resource at once to make progress.
- A fork could just as easily be a lock, a tape drive, or a memory buffer — the dining table is a dressed-up resource-allocation graph.
- That is exactly why this problem belongs right before the Deadlocks unit.

THE NAIVE SOLUTION

Pick Up Left, Then Right

```
semaphore fork[5] = {1, 1, 1, 1, 1};

/* Philosopher i */
do {
    wait(fork[i]);           /* pick up left fork */
    wait(fork[(i + 1) % 5]); /* pick up right fork */
    /* eat */
    signal(fork[i]);
    signal(fork[(i + 1) % 5]);
    /* think */
} while (true);
```

All Five Picking Up Their Left Fork at Once

ALL FIVE PICKING UP THEIR LEFT FORK AT ONCE DEADLOCKS THE TABLE

Suppose all five philosophers become hungry at roughly the same moment, and each executes `wait(fork[i])` — picking up their left fork — before any reaches `wait(fork[(i+1) % 5])`.

Every fork is now held: philosopher i holds `fork[i]`. Every philosopher then tries to pick up their right fork — precisely the left fork of their neighbor, already held by that neighbor.

All five block forever on `wait()`: P_0 waits on the fork held by P_1 , P_1 waits on the fork held by P_2 , ..., P_4 waits on the fork held by P_0 — a complete cycle.

This is a textbook circular-wait deadlock, and circular wait is one of the four conditions the Deadlocks unit names as necessary for deadlock to occur at all.

FIX 1: BREAKING THE SYMMETRY

Even Philosophers Reach Right First

```
/* Philosopher i */
do {
    if (i % 2 == 0) {
        wait(fork[(i + 1) % 5]); /* right first */
        wait(fork[i]);           /* then left  */
    } else {
        wait(fork[i]);           /* left first  */
        wait(fork[(i + 1) % 5]); /* then right */
    }
}
```

Even Philosophers Reach Right First

```
        /* eat */  
    signal(fork[i]);  
    signal(fork[(i + 1) % 5]);  
    /* think */  
} while (true);
```

Why Reversing Half the Philosophers Works

- The deadlock depended on every philosopher reaching for forks in the same rotational direction, so each grabs exactly one fork and then blocks on the next — a perfect, symmetric cycle.
- Reversing the order for half the philosophers destroys that symmetry: somewhere around the table, two adjacent philosophers now reach for the SAME fork as their first choice.
- Because acquiring a fork is atomic, only one of those two can actually win it — the other comes away holding nothing at all.
- A philosopher holding nothing can never be a link in a circular-wait chain, which is exactly what breaks the cycle.

Tracing the Worst Case

- P0 and P1 both reach for fork[1] first — one wins it, the other blocks holding nothing; P2 and P3 both reach for fork[3] first — same resolution.
- Whichever of P1/P3 wins its first fork goes on to acquire its second fork uncontested and eats; once it finishes and releases both forks, the philosophers it was blocking can proceed in turn.
- Every philosopher eventually eats — no cycle of mutual, unresolvable waiting can form.

Fix 2: The Monitor's Solution

- A monitor can express the problem's real constraint — a philosopher may pick up its forks only if neither neighbor is currently eating — directly.
- It uses an explicit state for each philosopher and a helper procedure, `test()`, that checks both neighbors atomically.

state[] and pickup()

```
monitor DiningPhilosophers {  
    enum {THINKING, HUNGRY, EATING} state[5];  
    condition self[5];  
  
    void pickup(int i) {  
        state[i] = HUNGRY;  
        test(i);  
        if (state[i] != EATING) { self[i].wait(); }  
    }  
}
```

putdown()

```
void putdown(int i) {  
    state[i] = THINKING;  
    test((i + 4) % 5);    /* left neighbor */  
    test((i + 1) % 5);    /* right neighbor */  
}
```

test() — the Heart of the Solution

```
void test(int i) {  
    if (state[(i + 4) % 5] != EATING &&  
        state[i] == HUNGRY &&  
        state[(i + 1) % 5] != EATING) {  
        state[i] = EATING;  
        self[i].signal();  
    }  
}  
  
initialization_code() {  
    for (int i = 0; i < 5; i++) { state[i] = THINKING; }  
}  
}
```

Where a Monitor Earns Its Keep

TEST(I) MOVES PHILOSOPHER I INTO EATING ONLY IF BOTH NEIGHBORS ARE NOT EATING

This check-and-update happens as one atomic step because the monitor itself guarantees mutual exclusion — no other philosopher's pickup(), putdown(), or test() can run concurrently and race against it.

When philosopher i finishes and calls putdown(i), it re-checks both neighbors via test(), so a neighbor that was waiting because i was eating gets woken the instant it is safe.

A pure-semaphore solution that is both deadlock-free and lets as many non-conflicting philosophers eat simultaneously as possible is notoriously fiddly to get right. The monitor expresses that exact rule in a few lines, bundling arbitrary state with an atomic check-and-act procedure — precisely what raw semaphores don't give you directly.

Dispatcher Objects

```
HANDLE hMutex = CreateMutex(NULL, FALSE, NULL);

WaitForSingleObject(hMutex, INFINITE); /* blocks until signaled */

    /* critical section */

ReleaseMutex(hMutex);                /* signals hMutex again */
```

A Common Abstraction Across Objects

- Windows exposes mutexes, semaphores, and events through a common abstraction called a dispatcher object. Every dispatcher object has a signaled or non-signaled state.
- `WaitForSingleObject()` blocks the calling thread until the given object becomes signaled (a free mutex, a posted semaphore, a set event) or a timeout elapses.
- `WaitForMultipleObjects()` extends this to wait on several dispatcher objects at once, either for any one or for all of them to become signaled.

Linux: Futexes and pthreads

- Linux's key primitive is the futex (“fast userspace mutex”): acquiring an uncontended lock is handled entirely in user space, via a single atomic compare_and_swap-style instruction, with no system call at all.
- The kernel is only involved — through the futex() system call — when there is actual contention: when a thread genuinely needs to block or needs to wake a blocked thread.
- Standard pthreads mutexes, semaphores (sem_wait()/sem_post()), and condition variables (pthread_cond_wait()/pthread_cond_signal()) are the usual application-facing API on Linux, built on top of a futex.

Same Idea, Two Real Implementations

Windows

Dispatcher objects (mutex, semaphore, event) +
`WaitForSingleObject()` / `WaitForMultipleObjects()`.

Linux

Futexes — user-space fast path, kernel only on contention —
underneath pthread mutexes, semaphores, condition variables.

Key Takeaways

- The Dining Philosophers Problem models processes contending for a fixed pool of resources where each process needs more than one resource simultaneously.
- The naive “left fork, then right fork” solution can deadlock: if every philosopher grabs their left fork at once, a complete circular-wait cycle forms and none can ever get a second fork.
- An asymmetric fork-ordering fix breaks that symmetry with no extra state: two neighbors contest the same fork as their first choice, and the loser holds nothing — which a circular-wait cycle cannot form around.
- The monitor-based solution expresses the real constraint — eat only if neither neighbor is eating — directly, using a `state[]` array and an `atomic test()` procedure.
- Windows unifies synchronization objects under dispatcher objects and `WaitForSingleObject()`; Linux optimizes the common, uncontended case with `futexes`, trapping into the kernel only on real contention.